kai: Day 1 Report

Lab cluster and read-only Kubernetes access

What was built, how it was built, what I learned, the mentor questions from every task, and the libraries used.

Project: kai, a Kubernetes troubleshooting agent in Go
Module: github.com/KanishkaVerma054/k8s-ai
Status of Day 1: done, "Done when" test passed

Contents

1. Overview: the goal of Day 1
2. Libraries and tools used
3. Architecture after Day 1
4. Task 1: Go module, cobra, client-go and a Makefile
5. Task 2: kind lab cluster and make lab
6. Task 3: the 8 broken scenarios
7. Task 4: internal/kube: kubeconfig loading and resource access
8. Task 5: describe-equivalent summaries, events, logs and kai get-context
9. Done-when test results
10. What I learned (Go, Kubernetes, agents, reading source)
11. Mentor questions asked in every task
12. Interview notes
13. Scope cuts and next steps

1. Overview: the goal of Day 1

kai is a Kubernetes troubleshooting agent written in Go. The 7-day plan builds it in layers: a read-only data layer (Day 1), deterministic analyzers (Day 2), an LLM agent loop (Day 3), safe mutations with approval (Day 4), more model providers (Day 5), evals and prompt-injection tests (Day 6), and MCP (Day 7).

Day 1 builds the foundation everything else stands on: a reproducible lab cluster with 8 deliberately broken apps, and Go code that reads everything relevant about a Deployment from the cluster, the way `kubectl describe`, `kubectl get events` and `kubectl logs` would, and prints it as one JSON document. No LLM is involved yet.

Done when: `make lab` brings up the cluster with all 8 scenarios, and `kai get-context deploy/checkout-service -n <scenario>` shows the real failure reason for each one.

Why the data layer comes first

- An agent is only as good as the evidence it is given. If the data collection misses the previous container's log or the PVC's events, no model can find the cause.
- Deterministic code is cheap, testable and repeatable. Seven of the eight scenarios are explained by a single field or event, without any model.
- Summaries keep the evidence and drop the noise, which saves tokens and stops the real cause from being buried (this matters on Day 3).

2. Libraries and tools used

Go libraries (go.mod)

Library	Version	What it is used for in Day 1
`k8s.io/client-go`	v0.37.2	The official Go client for the Kubernetes API. `kubernetes.Clientset` and `kubernetes.Interface` (typed clients for every API group), `tools/clientcmd` (kubeconfig loading, contexts, in-cluster fallback), `kubernetes/fake` (in-memory cluster for tests).
`k8s.io/api`	v0.37.2	Go types for Kubernetes objects: `core/v1` (Pod, Service, PVC, Event, PodLogOptions), `apps/v1` (Deployment), `networking/v1` (Ingress).
`k8s.io/apimachinery`	v0.37.2	Shared machinery: `meta/v1` (ObjectMeta, ListOptions, LabelSelectorAsSelector, Time), `fields` (field selectors for events).
`github.com/spf13/cobra`	v1.10.2	The CLI framework: root command, the `get-context` subcommand, persistent flags (`--kubeconfig`, `--context`, `-n`), argument validation, `ExecuteContext`.

Go standard library packages

Package	Used for
`context`	Cancellation and deadlines on every API call (`context.WithTimeout`).
`os/signal`	`signal.NotifyContext` cancels the context on Ctrl+C.
`encoding/json`	Indented JSON output with struct tags and `omitempty`.
`errors`, `fmt`	Sentinel error `ErrLogsUnavailable`, `errors.Is`, wrapping with `%w`.
`slices`	`slices.SortStableFunc` to sort events newest first.
`strings`, `bytes`	Parsing `deploy/<name>` with `strings.Cut`; detecting the kubelet's "unable to retrieve container logs" text.
`time`	Event timestamps and the `--timeout` flag.
`testing`	Table-driven tests with `t.Run`.

Tools

Tool	Role
Go 1.26 toolchain	`go build`, `go vet`, `go test`, `go mod tidy`, `go doc`, `gofmt`.
golangci-lint	Extra static checks (it caught an unchecked `fmt.Fprintf` via errcheck).
kind (`kindest/node:v1.37.0`)	Runs the local `kai-lab` Kubernetes cluster inside Docker.
kubectl	Applying the scenarios and comparing kai's output with `describe` / `logs`.
Docker	Hosts the kind node container.
Make	`make build`, `make test`, `make lab`, `make lab-down`, `make lab-reset`.

3. Architecture after Day 1

```
kai get-context deploy/checkout-service -n crash-missing-env --context kind-kai-lab
     |
     v
cmd/kai/main.go          root command, persistent flags, Ctrl+C -> cancel ctx
cmd/kai/getcontext.go    parseDeployTarget -> ResolveTarget (prints context/ns to stderr)
     |                   -> NewClientset -> WithTimeout(30s) -> GetContext -> JSON to stdout
     v
internal/kube/client.go     Options{Kubeconfig, Context}, loader(), NewClientset, ResolveTarget
internal/kube/resources.go  Client{cs kubernetes.Interface}, New, GetDeployment,
                            ListPodsForDeployment, ListServices, ListPVCs, ListIngresses, ListEvents
internal/kube/summary.go    SummarizePod -> PodSummary / ContainerSummary / StateSummary
internal/kube/events.go     ListEventsFor (field selector), sort newest first, merge duplicates
internal/kube/logs.go       GetLogs (current / previous, line + byte caps, ErrLogsUnavailable)
internal/kube/context.go    GetContext: Deployment + pods + events + logs + Services + PVCs + Ingresses
     |
     v
Kubernetes API server (kind cluster kai-lab)   or   fake.NewClientset(...) in tests
```

Key design decisions

- **Accept interfaces, return structs.** `NewClientset` returns the concrete `*kubernetes.Clientset`; `kube.Client` stores a `kubernetes.Interface`, so tests inject `fake.NewClientset(...)` and need no cluster.
- **Create the client once at the edge** (in the command) and pass it in: dependency injection. Methods use `c.cs`, never build their own client.
- **One shared** `loader()` for both the client and `ResolveTarget`, so the context kai prints is guaranteed to be the one it talks to.
- **Explicit** `--context`**.** kai prints `kai: context X, namespace Y` on stderr before doing anything, because the current kubectl context once silently pointed at the wrong cluster.
- **Only a missing Deployment is fatal.** Every other failure (a log, a PVC, events) goes into the `errors` list and the rest is still returned: partial results beat no results.
- **stdout is data, stderr is messages**, so `kai get-context ... | jq` works.

4. Task 1: Go module, cobra, client-go and a Makefile

What was built

- `go.mod` with cobra v1.10.2 and client-go / api / apimachinery v0.37.2.
- `cmd/kai/main.go` with `newRootCmd()` and `main()`.
- A Makefile with `build`, `run` and `test` targets, and a `.gitignore` (`.env`, `bin/`, `kubeconfig*`, `/kai`).

How it was done

`newRootCmd()` builds a fresh cobra command every time it is called; `main()` runs it and exits with status 1 on error, because `main` cannot return an error. The client-go version was chosen as v0.37.2 (Kubernetes 1.37) instead of matching the old local kubectl 1.32, since 1.32 is out of upstream support; this raised the `go` directive to 1.26.0 and meant upgrading kind and kubectl.

Problems and fixes

- `package kai` built but would not run ("is not a main package"): a program must be `package main`.
- `go build ./...` dropped a `kai` binary in the repo root, so `/kai` went into `.gitignore`.
- Review: `newRootCmd` changed from a global variable to a function so state cannot leak between tests; build `./cmd/kai` instead of `main.go`; add `.PHONY` and a `test` target.

5. Task 2: kind lab cluster and make lab

What was built

`lab/kind.yaml`, a single-node cluster pinned to `kindest/node:v1.37.0` by digest, and the Makefile targets `lab`, `lab-down` and `lab-reset`.

How it was done

- `make lab` is idempotent: it creates `kai-lab` only if it does not exist, waits for the node to be Ready, then applies the scenarios.
- Every kubectl command passes `--context kind-kai-lab`, so it can never hit another cluster.
- The cluster name lives in one Makefile variable; destroying is a separate, deliberate command (`lab-down`).
- The node image is pinned by `sha256` digest so the exact version can never change under you.

Problems and fixes

- `make: kind: No such file or directory` (exit 127): `~/go/bin` was not on `PATH`.
- Cluster creation timed out with a misleading `ClusterRoleBinding ... context deadline exceeded`. Logs showed the control plane needed about 47 s of kubeadm's roughly 60 s, so the kubeadm timeouts were raised to 5 minutes (from 1/7 to 2/2 successes).
- The node container showed `Exited (137)` with `OOMKilled`. `docker events` showed a `stop` instead, and the flag came from scenario 4's expected OOM kills. Lesson: exit 137 only means SIGKILL, so verify before trusting a flag.

6. Task 3: the 8 broken scenarios

Each scenario has its own namespace and a `checkout-service` Deployment with exactly one planted bug. Separate namespaces keep events apart and make reset a single `kubectl delete namespace`. Every app runs correctly once its bug is fixed, so a correct fix on Day 4 really makes the pod healthy. Together they form the eval set for Day 6.

#	Namespace	Planted bug	Where the evidence is
1	`crash-missing-env`	`DATABASE_URL` never set	Logs (CrashLoopBackOff)

#	Namespace	Planted bug	Where the evidence is
2	`crash-missing-config-key`	ConfigMap key `user` missing	Events (CreateContainerConfigError)
3	`imagepull-bad-tag`	Image `busybox:1.36-typo`	Events (ImagePullBackOff)
4	`oomkilled-low-memory`	Needs ~100 MB, limit 32Mi	Last termination OOMKilled, exit 137, plus spec limit
5	`liveness-wrong-port`	App on 8080, probe on 9090	Events + probe vs container port in spec
6	`pending-too-large`	Requests 1000 CPUs and 1Ti	Events (FailedScheduling)
7	`pvc-missing-storageclass`	StorageClass `fast-ssd` does not exist	The PVC's events
8	`service-selector-mismatch`	Service selects `app=checkout`	Service selector vs pod labels

Problems and fixes

- VS Code matched `scenarios/` to an unrelated CrowdSec schema; mapped `lab/scenarios/**/*.yaml` to the kubernetes schema.
- First versions of scenarios 2 and 3 had no `command`, so busybox exited immediately and would never be healthy even after a fix.
- Scenario 2 is `CreateContainerConfigError`, not `CrashLoopBackOff`: the container never starts, so it has no logs.
- A changed Deployment left two pods side by side (stuck rollout), which taught that a Deployment can have several pods at once.

7. Task 4: internal/kube: kubeconfig loading and resource access

What was built

- `client.go`: `Options{Kubeconfig, Context}`, a shared `loader()`, `NewClientset(opts)` and `ResolveTarget(opts)`.
- `resources.go`: `Client{cs kubernetes.Interface}`, `New(cs)`, `GetDeployment`, `ListPodsForDeployment`, `ListServices`, `ListPVCs`, `ListIngresses`, `ListEvents`.

How it works

```
func loader(opts Options) clientcmd.ClientConfig {
    rules := clientcmd.NewDefaultClientConfigLoadingRules()   // $KUBECONFIG, then ~/.kube/config
    if opts.Kubeconfig != "" {
        rules.ExplicitPath = opts.Kubeconfig                  // only when the user gave one
    }
    overrides := &clientcmd.ConfigOverrides{CurrentContext: opts.Context}  // keyed literal
    return clientcmd.NewNonInteractiveDeferredLoadingClientConfig(rules, overrides)
}
```

- client-go reads the kubeconfig first and falls back to in-cluster config when no kubeconfig is found. It detects "inside a pod" from `KUBERNETES_SERVICE_HOST`, `KUBERNETES_SERVICE_PORT` and the service-account token file.
- `ListPodsForDeployment` turns `Spec.Selector` into a string such as `app=checkout-service` with `metav1.LabelSelectorAsSelector(...).String()`, so the API server does the filtering and a stuck rollout returns all its pods.
- Every method takes `ctx` first, wraps errors with the step and `ns/name` (`list pods for deployment shop/checkout: ...`) using `%w`, and returns `list.Items`.
- Services, PVCs and Ingresses are listed for the whole namespace, because the selector lives on the Service and points at pods, not the other way round.

Problems and fixes

Error	Underlying concept
`missing go.sum entry`	Nothing had imported client-go yet; `go mod tidy` records checksums.
`too many arguments` in `NewDefaultClientConfigLoadingRules(path)`	Arguments vs struct fields: it takes none; set `ExplicitPath` on the result.
`cannot use opts.Context as api.AuthInfo`	Positional struct literals fill fields in order; use keyed literals.
`fmt.Errorf` without `return`	Building an error does nothing unless it is returned.
`v1 redeclared`	Packages are named by the last path element; alias imports (`corev1`, `appsv1`, ...).
`import cycle not allowed`	A package never imports itself; inside it use `c.cs` / `New(...)`.
Reusing `d` for the selector	`:=` / `=` can overwrite a parameter; use a new name `sel`.
`assignment mismatch: 1 variable but ... returns 2 values`	Go makes you receive every return value.
`sel.string()`	Lowercase is unexported; the method is `String()`.
`go vet`: arguments but no formatting directives	Each `%` verb takes exactly one value.
11 errors from one `ListOptions{ListServices: }`	After a syntax error, fix only the first error and rebuild.

8. Task 5: describe-equivalent summaries, events, logs and kai get-context

What was built

- `summary.go`: `SummarizePod` (a pure function) producing `PodSummary`, `ContainerSummary`, `StateSummary`, with `LastTermination *StateSummary`.
- `events.go`: `ListEventsFor(ns, kind, name)` with a field selector, `SortEventsNewestFirst`, `mergeDuplicateEvents`, and `eventTime`, which picks the best of five timestamp fields.
- `logs.go`: `GetLogs` for current or previous logs, capped at 50 lines and 16 KiB on the server side, with the sentinel `ErrLogsUnavailable`.
- `context.go`: `GetContext` returns a `DeploymentContext` with the Deployment summary (replicas, conditions, container spec: image, requests/limits, ports, probes), pods with events and logs, Services (`selects_deployment_pods`), PVCs with events, Ingresses, and `errors`.
- `cmd/kai/getcontext.go`: the `get-context deploy/<name>` command with `--log-lines` and `--timeout`; `main.go` gained `--kubeconfig`, `--context`, `-n` and Ctrl+C cancellation.
- Tests: `summary_test.go`, `context_test.go` (fake clientset), `getcontext_test.go` (table test for `parseDeployTarget`).

How it works

- `ContainerState` has three pointers (`Waiting`, `Running`, `Terminated`) and only one is set; `summarizeState` switches over them and every pointer is checked before it is read.
- A pointer in kai's own struct (`LastTermination`) makes "never terminated" different from "exited 0".
- Current logs are skipped for a container that is waiting with 0 restarts (it never ran); previous logs are read only when there was a last termination.
- Duplicate Event objects with the same type, reason, message and object are merged and their counts added (scenario 7 had up to 9 copies, 562 in total).
- The fake clientset ignores field selectors, so `ListEventsFor` filters again in Go, and a test covers it.
- Struct embedding (`PodContext` embeds `PodSummary`) flattens fields into one JSON object; `omitempty` drops empty lists.

Problems and fixes

- Scenario 1's previous log came back as the text `unable to retrieve container logs` with a success status: right after a crash, "previous" points at a container the kubelet already deleted. `GetLogs` now turns it into `ErrLogsUnavailable`, checkable with `errors.Is`.
- A test expected timestamps rounded to seconds, but the fake keeps full precision (only the real API server rounds).
- golangci-lint (errcheck) flagged an unchecked `fmt.Fprintf`; switched to cobra's `cmd.PrintErrf`.

Reading the output: scenario 1

```
"phase": "Running",                                  <- misleading on its own
"restart_count": 165,
"state":            {"state": "waiting", "reason": "CrashLoopBackOff"},
"last_termination": {"state": "terminated", "reason": "Error", "exit_code": 1},
"logs": [{"container": "app",
          "current":  "FATAL: DATABASE_URL is not set\n",
          "previous": "FATAL: DATABASE_URL is not set\n"}]   <- the root cause
```

The Deployment shows the symptom (0/1 ready, `MinimumReplicasUnavailable`), the container state shows the crash pattern, exit code 1 means the app exited by itself (compare 137 = killed by the kernel in scenario 4), and only the logs name the cause.

9. Done-when test results

Passed. `make lab` reported every scenario unchanged (all 8 running), and `kai get-context` showed the real failure reason for each, without an LLM:

Scenario	Evidence kai printed
`crash-missing-env`	`FATAL: DATABASE_URL is not set` (previous log)
`crash-missing-config-key`	`Error: couldn't find key user in ConfigMap ...` (event)
`imagepull-bad-tag`	`Failed to pull image "busybox:1.36-typo": ... NotFound` (event)
`oomkilled-low-memory`	`OOMKilled`, exit 137, limit 32Mi (last termination + spec)
`liveness-wrong-port`	`Liveness probe failed: Get "http://...:9090/"` (probe 9090 vs port 8080)
`pending-too-large`	`0/1 nodes are available: 1 Insufficient cpu, 1 Insufficient memory`
`pvc-missing-storageclass`	`storageclass.storage.k8s.io "fast-ssd" not found` (PVC event)
`service-selector-mismatch`	selector `{app: checkout}`, `selects_deployment_pods=false`

`go vet`, `golangci-lint` (0 issues) and `go test ./...` all pass.

Biggest lesson: each failure leaves its evidence somewhere different (logs, events, last termination, the spec, a related PVC or Service), so a troubleshooting tool has to collect all of it.

10. What I learned

Go

- The module path prefixes every import; `internal/` packages cannot be imported by other projects (the compiler enforces it).
- The `go` line in `go.mod` is a minimum version; `go.sum` stores checksums; `go mod tidy` adds them and removes `// indirect` from modules imported directly.
- `package main` + `func main()` is a program; `main` cannot return an error, so failure is `os.Exit(1)`.
- Unused imports and variables are compile errors. Read errors as `file:line:column: message`; `have (...)` / `want (...)` compares what was passed with what is expected.
- Use keyed struct literals. Every `return` gives every value: `nil, err` on error, `value, nil` on success. The caller checks `err` first.
- `fmt.Errorf` only builds an error, so it must be returned. `%w` wraps the cause so `errors.Is` still works. Error messages are lowercase and name the step.
- Interfaces are method sets satisfied implicitly. "Accept interfaces, return structs" makes code testable with fakes.
- Import aliases avoid clashes between `v1` packages; a package never imports itself; import cycles are forbidden.
- Methods have receivers (`func (c *Client) ...`); exported names are uppercase, private ones lowercase.
- `:=` needs at least one new variable and can shadow or overwrite by accident. Count the `%` verbs in format strings.
- Pure functions are the easiest to test. Pointer fields mean "may be absent". JSON tags and `omitempty` shape the output. Struct embedding promotes fields.
- Sentinel errors (`var ErrX = errors.New(...)`) wrapped with `%w` are checked with `errors.Is`.
- `context.WithTimeout` puts a deadline on every call that uses the ctx; `signal.NotifyContext` cancels on Ctrl+C.

- Table-driven tests with `t.Run`; `fake.NewClientset(objects...)` gives an in-memory cluster.
- Duplication is only a problem when the same decision is copied. Generics cannot read a field like `.Items` from a type parameter, so the four List functions stay separate (rule of three).

Kubernetes

- client-go `v0.37.x` maps to Kubernetes 1.37; keep all three `k8s.io` modules on the same version.
- Always pass `--context` explicitly; the current context can point at the wrong cluster.
- A Deployment does not list its pods; `Spec.Selector` says which labels they carry. A stuck rollout keeps several pods at once.
- A Service's selector points at pods, so kai lists all Services and checks which should match.
- Pod phase is coarse: a crashlooping pod still shows `Running`. The truth is in container states, restart count and last termination.
- `STATUS` in `kubectl get pods` is a snapshot that cycles; `Unknown` means the kubelet has not reported yet.
- Label selectors filter on labels, field selectors on fields (`involvedObject.kind/name`). Both make the API server do the filtering.
- Events have several timestamp fields filled by different components, and the same failure is often stored as many Event objects.
- API groups match `apiVersion`: Pods, Services, PVCs, Events are `CoreV1()`; Deployments `AppsV1()`; Ingresses `NetworkingV1()`.
- Exit 137 means SIGKILL (often OOMKilled); exit 1 means the app exited by itself.
- In a pod, Kubernetes sets `KUBERNETES_SERVICE_HOST` / `PORT` and mounts a service-account token; that is how client-go detects in-cluster mode. A 403 with no credentials means `system:anonymous` was refused by RBAC.

Agentic AI

- Deterministic data collection comes before the LLM: cheap, testable, repeatable. The model is for explaining and for ambiguous cases.
- The JSON from `get-context` is what the Day 2 analyzers and the Day 3 model will read, so it keeps only evidence: capped logs, merged events, summaries instead of raw objects.
- Partial failures are reported, not fatal, so the agent still gets what could be read (important with RBAC on Day 4).
- The 8 scenarios are the eval set: checking the status is not enough, the answer must name the exact bug.
- A reproducible environment (pinned images, idempotent `make lab`) makes eval scores comparable between runs.

Reading library source

- Find the exact downloaded version: `go list -m -f '{{.Dir}}' k8s.io/client-go`.
- Ways in: F12 / Ctrl+F12 in VS Code, `go doc` and `go doc -src`, `grep -rn` in the module folder, pkg.go.dev.
- Start with one question, read the doc comment then the code, go one level deep, check tests and examples, and stop once you can point at the file and line that answers it.
- Example: client-go ignores an empty `ExplicitPath` (`loader.go:208`), and the in-cluster fallback lives in `merged_client_builder.go`.

11. Mentor questions asked in every task

These are the questions asked before, during and after each task, to make me think before getting an answer. Answers are short versions of what we concluded.

Task 1: Go module, cobra, client-go, Makefile

Q Layout: the plan puts `main` in `cmd/kai/` and everything else in `internal/`. What does Go enforce about `internal/`, and why would you want that?

> Answer: Only code inside the same module tree can import it, so kai's internals are not a public API that others can depend on.

Q client-go versions map to Kubernetes versions (`v0.32.x` is 1.32). Which version would you pick, and what could go wrong if it is far from the cluster's version?

> Answer: v0.37.2: 1.32 is out of support. A large skew can mean missing or changed API fields. Check support, requirements and compatibility, not just "match what's installed" or "use the latest".

Q `Execute()` returns an error. Why can't `main()` just `return err`, and how does the program report failure to the shell?

> Answer: `main` has no return value; report failure with `os.Exit(1)`.

Q Why is it `newRootCmd().Execute()` now, when it was `newRootCmd.Execute()` before?

> Answer: It is now a function that builds a fresh command each call, not a shared global variable, so state cannot leak between tests.

Task 2: kind cluster and make lab

Q Number of nodes: one node, or a control plane plus workers? Do any of the 8 scenarios need a second node?

> Answer: One node: no scenario needs a second, and it starts faster and uses less memory.

Q `kind create cluster` fails if `kai-lab` already exists. Should `make lab` fail, skip, or delete and recreate?

> Answer: Skip: the command should be idempotent; destroying is a separate, deliberate command.

Q Your kubeconfig has other clusters and a stale current context. How will the Makefile make sure `kubectl apply` can only ever hit `kai-lab`?

> Answer: Pass `--context kind-kai-lab` on every kubectl command.

Q Why pin the node image with a `sha256` digest as well as the `v1.37.0` tag?

> Answer: A tag can be moved; a digest identifies exactly one image, so the lab is reproducible.

Task 3: the 8 broken scenarios

Q Isolation: should all 8 scenarios share one namespace, or should each get its own?

> Answer: One namespace each: events do not mix, and `kubectl delete namespace` resets a scenario.

Q Where does the failure show for OOMKilled, ImagePullBackOff and a Service with no endpoints: `get`, `describe`, `logs` or `logs --previous`?

> Answer: OOMKilled: Last State / exit 137 in `describe`. ImagePullBackOff: events in `describe`. No endpoints: the Service selector vs pod labels.

Q Who writes the YAML: you write the first two and I write the rest, or you write them all?

> Answer: I wrote scenarios 2 and 3; the rest were written for me and walked through.

Q What status do you expect each scenario to show in `kubectl get pods`?

> Answer: e.g. CrashLoopBackOff, CreateContainerConfigError, ImagePullBackOff, OOMKilled, Pending; `STATUS` is a snapshot that cycles.

Q For scenario 2, will `kubectl logs` show anything? Why or why not?

> Answer: No: the container never started (CreateContainerConfigError), so the cause is only in the events.

Task 4a: kubeconfig loading (client.go)

Q Order: should kai try in-cluster config or the kubeconfig first? How can it tell it is running inside a pod?

> Answer: Kubeconfig first, then in-cluster, like kubectl. Inside a pod Kubernetes sets `KUBERNETES_SERVICE_HOST`, `KUBERNETES_SERVICE_PORT` and mounts a service-account token.

Q Choosing the cluster: always use the current context, or accept a `--context` flag like kubectl?

> Answer: A `--context` flag; empty means the current context.

Q Return type: `*kubernetes.Clientset` or `kubernetes.Interface`? Which lets the rest of the code work with both the real and the fake client?

> Answer: Return the struct; accept `kubernetes.Interface` everywhere it is used.

Q In-cluster config has no context name. What should kai do if someone runs it in a pod with `--context kind-kai-lab`?

> Answer: client-go ignores the context in in-cluster mode; `ResolveTarget` reports "in-cluster" so the user can see it.

Q On an error the function must `return` immediately. It returns two things: what is each one on failure, and on success?

> Answer: `nil, err` on failure; `clientset, nil` on success.

Q Why does each error path return `nil` as the first value?

> Answer: There is no valid client to give back, and the caller must not use it.

Q Why is `ExplicitPath` set inside an `if`, instead of always set to `opts.Kubeconfig`?

> Answer: Only override when the user gave a path; otherwise keep the default `$KUBECONFIG` / `~/.kube/config` search.

Q If you run `kai --context kind-kai-lab`, which line makes sure kai talks to that cluster?

> Answer: `ConfigOverrides{CurrentContext: opts.Context}`.

Task 4b: resource access (resources.go)

Q Shape: plain functions that each take the client, like `ListPods(ctx, client, ns)`, or methods on a small struct that holds the client? What does each mean for tests?

> Answer: Methods on `Client`, which holds a `kubernetes.Interface`, so tests build it once with a fake clientset.

Q What would break if the field's type were `*kubernetes.Clientset` instead of `kubernetes.Interface`?

> Answer: Tests could not pass `fake.NewClientset(...)`, so every test would need a real cluster.

Q Why does `GetDeployment` use `c.cs`, and not call `NewClientset`?

> Answer: The client is created once at the edge and injected; building one inside would ignore `--context` and break tests (and caused an import cycle).

Q If someone runs kai with the wrong namespace, what will the error look like, and which code builds it?

> Answer: `get deployment wrong-ns/checkout-service: ... not found`, built by the `fmt.Errorf("get deployment %s/%s: %w", ...)` wrap.

Q Run `go mod tidy` again: why do `k8s.io/api` and `apimachinery` lose their `// indirect` marker?

> Answer: My code now imports them directly.

Q ListServices: which namespace argument, and what goes in `metav1.ListOptions{...}` when you don't want to filter? What does `List` return and what inside it do you return?

> Answer: `ns`; empty `ListOptions{}`; it returns a `*ServiceList`, and you return `list.Items`.

Q Why list all Services instead of filtering?

> Answer: The selector is on the Service and points at pods, so kai must compare every Service's selector with the pod labels.

Q The four List functions look alike. What would merging them take, would it be easier to read, and how often will they change?

> Answer: Generics plus an adapter per type (a type parameter cannot read `.Items`). It would be harder to read, and they rarely change, so keep them (rule of three).

Task 5: summaries, events, logs, get-context

Q Which fields from `kubectl describe` would you put in a `PodSummary`, and which would you leave out?

> Answer: Phase, node, conditions, and per container image, ready, restarts, state and last termination; leave out managed fields, UIDs, tolerations and other noise.

Q The three-pointer problem: `ContainerState` has `Waiting`, `Running`, `Terminated`, and only one is set. What happens if code reads `state.Terminated.ExitCode` when `Terminated` is nil? How do you guard it?

> Answer: A nil-pointer panic; check each pointer (a `switch` over them) before reading.

Q What happens if the pod has no previous container?

> Answer: `LastTerminationState.Terminated` is nil, so kai skips the previous log; and if the kubelet already removed it, `GetLogs` returns `ErrLogsUnavailable` instead of fake log text.

Q Is the output correct for scenario 1, and which field proves the root cause?

> Answer: Yes: `logs.previous` says `FATAL: DATABASE_URL is not set`; `phase: Running` alone is misleading.

12. Interview notes

Q Walk me through kai's architecture.

> Day 1 is the read-only data layer: `NewClientset` (kubeconfig or in-cluster) -> `kube.Client` wrapping a `kubernetes.Interface` -> small functions that each fetch one thing -> `GetContext` combining them into one JSON document. Analyzers and the agent sit on top and never call client-go directly.

Q Why `kubernetes.Interface` and not `*Clientset`?

> Accept interfaces, return structs: tests pass `fake.NewClientset(...)` and need no cluster.

Q How do you avoid talking to the wrong cluster?

> `--context` on every command, one shared loader for the client and the printed context, and kai prints `context X, namespace Y` first.

Q Why summaries instead of raw objects?

> A pod has hundreds of fields and its phase can say `Running` while it crashloops. Summaries keep the evidence and cut tokens.

Q How do you handle nil pointers in Kubernetes types?

> Check every pointer before reading it, and use pointers in your own types so "absent" is different from "zero".

Q What happens when part of the data can't be read?

> Fail only on what makes the answer meaningless (missing Deployment); record the rest in `errors` and continue.

Q Why cap logs?

> One chatty container can produce megabytes; `TailLines` and `LimitBytes` cap them on the server side.

Q Tell me about a tricky bug.

> `logs --previous` returned "unable to retrieve container logs" as log text with a success status, right after a crash. I turned it into the sentinel error `ErrLogsUnavailable`, checked with `errors.Is`.

Q Why server-side filtering?

> Label and field selectors make the API server filter, so kai does not download whole namespaces. The fake ignores field selectors, so the code double-checks and a test covers it.

Q Why not merge the List functions with generics?

> A type parameter cannot read `.Items`; the generic version would be longer and harder to read. Rule of three.

Q Why deterministic collection before the LLM?

> Cheap, testable, repeatable. Seven of eight scenarios are explained by one field or event without a model.

13. Scope cuts and next steps

Scope cuts

- Init containers and ReplicaSet events are not collected yet.
- EndpointSlices are not read; `selects_deployment_pods` is computed from labels.
- The default namespace comes from the kubeconfig context; there is no `--all-namespaces`.
- No unit test for `ErrLogsUnavailable`, because the fake clientset always returns `fake logs`.

Still to do myself

- Read `summary.go`, `events.go`, `logs.go` and `context.go` (written at my request) and be able to explain them line by line.
- Commit Day 1 in small commits and push.

Next: Day 2, deterministic analyzers

Turn each "one field that explains the bug" from the done-when table into an analyzer that reads the `DeploymentContext` and reports a finding, with table-driven tests. The LLM comes on Day 3.